

Security in practice: Cryptography exercises with OpenSSL library

Diana Berbecaru
diana.berbecaru@polito.it

Course master in e-business and ICT security (2011)

OpenSSL

- SSLeay: developed in 1995 by Eric A. Young and Tim J. Hudson; from 1998 takes the name OpenSSL (0.9.1c).
 - current (stable) version: 1.0.0d (released 08 Feb 2011)
- downloadable (sources and binaries) from:
<http://www.openssl.org/source/>
- open source library, composed of two libraries:
 - library of crypto functions (*libcrypto*)
 - library used for the management of SSL protocol (*libssl*)
- the main characteristic of this library is the complete implementation of SSLv2, SSLv3 and TLSv1 protocols
- other crypto libraries: Crypto++, Cryptlib, BouncyCastle

Course master in e-business and ICT security

2011

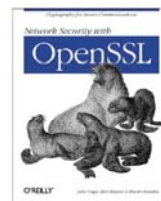
Documentation

- not very much, but the situation is getting better:
 - the most updated: <http://www.openssl.org/docs/>
 - books and articles:
 - Eric Rescorla: *SSL and TLS: Designing and Building Secure Systems*, Addison-Wesley, 2001 - per la parte di SSL/TLS, (<http://www.rtfm.com/>)
 - John Viega, Matt Messier, and Pravir Chandra: *Network Security with OpenSSL: Cryptography for Secure Communications*, Giugno 2002, esempi al URL: <http://www.opensslbook.com/>
 - Stephen Thomas: *SSL and TLS Essentials, Securing the Web*, John Wiley & Sons, 2000
 - Eric Rescorla: *An Introduction to OpenSSL Programming (Part I and II)*, 2001-2002, (<http://www.rtfm.com/openssl-examples/>), <http://www.linuxjournal.com/article/4822>
 - Girish Venkatachalam: *Security Programming with OpenSSL*, March 30th, 2006, <http://www.linuxjournal.com/article/8756>
- mailing list: <http://www.openssl.org/support/>
- the code of demo applications is distributed with the library itself !!!
- the file openssl.txt in the directory doc

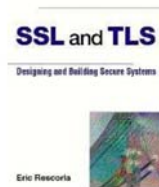
Course master in e-business and ICT security

2011

Useful books



Network Security with OpenSSL.



SSL and TLS: Designing and Building Secure Systems

Course master in e-business and ICT security

2011

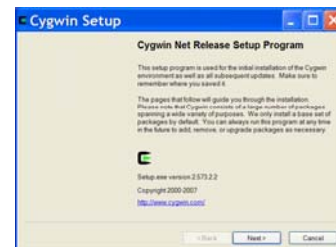
Installing OpenSSL (1)

- typically, by downloading the version from the official web site (<http://www.openssl.org/>).
 - for obvious security reasons it is always advisable to get the last available openssl version, to update it periodically and to compile it independently on your machine; some distributions of binary files, typically precompiled, are not provided by the developers of OpenSSL but by third parties (for example Win32 OpenSSL disponibile available at : <http://www.slproweb.com/products/Win32OpenSSL.html>)
- on Win platform it is possible to install it together with Cygwin Linux-like environment for Windows:
 - <http://www.cygwin.com>

Course master in e-business and ICT security

2011

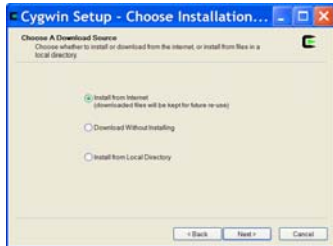
Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

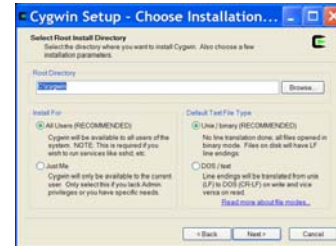
Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

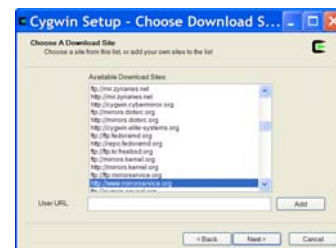
Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

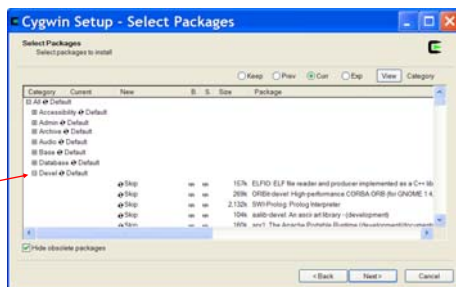
Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

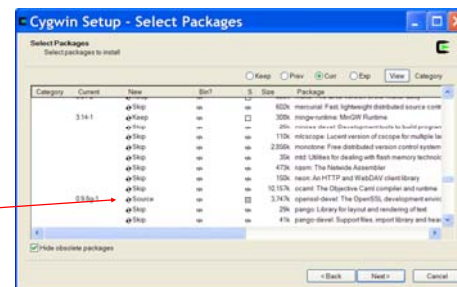
Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

Installing OpenSSL – Setup of Cygwin



Course master in e-business and ICT security

2011

Overview of Openssl features

- symmetric block algorithms: AES (from openssl version 0.9.7) DES, 3DES, DESX, CAST, RC2, RC5, IDEA, Blowfish
 - in modes CBC, CFB, ECB and OFB; for each cipher the default mode is CBC
- symmetric algorithms of type stream: RC4
- hash algorithms: MD2, MD4, MD5, SHA-1, RIPEMD 160, MDC2
- asymmetric algorithms: RSA, DSA, DH, ECC
- authentication: HMAC
- digital certificates and SSL: X509, X509v3, SSLv3, TLSv1
- Input/Output, formats: asn1, bio, evp, pem, pkcs7, pkcs12
- internal functions: bn, buffer, lhash, object, stack

Course master in e-business and ICT security

2011

Command-line interface of OpenSSL

- OpenSSL can be used by application developers to include cryptographic support into their applications ...
- ...but it can be used also as command-line security tool (called *openssl* on Linux and *openssl.exe* on Windows)
 - the tool allows users to use the characteristics of the library from the command line, e.g. calculate the hash of a message, or encrypt/decrypt – sign/verify with symmetric and asymmetric cryptography respectively.
 - used in "interactive" or "batch" modes

Course master in e-business and ICT security

2011

Interactive Mode

- by executing *openssl* (with no option), the tool enters in "interactive" mode; a prompt indicates that the program is ready to process standard openssl commands:

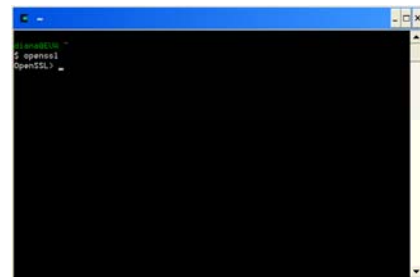
```
$ openssl
OpenSSL > standard openssl commands
```

- when the execution of a command is completed, the prompt reappears, indicating that the tool is ready to execute another openssl command
- you can go out from the command-line tool by executing: *quit*

Course master in e-business and ICT security

2011

Interactive Mode



Course master in e-business and ICT security

2011

Batch Mode

- by executing *openssl* (with options); similar to the "interactive" mode, except that for each openssl command it must be added *openssl* in the front

```
$ openssl standard openssl commands
```

- syntax of standard openssl commands

```
name command [options]
```

- in general (with some exception), the order of options is not important

Course master in e-business and ICT security

2011

Standard openssl commands (1)

- *asn1parse*
 - parses a ASN.1 sequence
- *base64*
 - encodes/decodes in/from Base 64 format
- *ca*
 - management of a test Certification Authority (CA)
- *ciphers*
 - description of a cipher suite
- *crl*
 - management of Certificate Revocation List (CRL)
- *crl2pkcs7*
 - conversion of CRL to PKCS#7

Course master in e-business and ICT security

2011

Standard openssl commands(2)

- **dgst**
 - computation of a message digest
- **dh**
 - management of Diffie-Hellman parameters. Obsolated by the command dhparam
- **dsa**
 - management of DSA data
- **dsaparam**
 - management of DSA parameters
- **enc**
 - encryption/decryption

Course master in e-business and ICT security

2011

Standard openssl commands(3)

- **errstr**
 - conversion of an error number to a string error
- **dhparam**
 - generation and management of Diffie-Hellman parameters
- **gendh**
 - generation of Diffie-Hellman parameters. obsolated by dhparam
- **gendsa**
 - generation of DSA parameters

Course master in e-business and ICT security

2011

Standard openssl commands (4)

- **genrsa**
 - generation of RSA parameters (e.g. keys)
- **ocsp**
 - tool for Online Certificate Status Protocol
- **passwd**
 - generations of hashed passwords
- **pkcs12**
 - management of data PKCS#12
- **pkcs7**
 - management of data PKCS#7

Course master in e-business and ICT security

2011

Standard openssl commands (5)

- **rand**
 - generation of pseudo-random numbers (bytes)
- **req**
 - management of requests of X.509 certificates - Certificate Signing Request (CSR)
- **rsa**
 - RSA data management
- **rsautl**
 - tool used to sign, verify, encrypt and decrypt data with RSA algorithm

Course master in e-business and ICT security

2011

Standard openssl commands(6)

- **s_server**
 - a generic SSL/TLS server that accepts remote connections from clients that support the security protocol SSL/TLS. This application has been written to test the functionality of OpenSSL and has a minimal interface; internally however it uses almost all functionalities of OpenSSL.
- **s_client**
 - a generic SSL/TLS client that establishes an SSL/TLS connection with a remote SSL/TLS server; minimal interface too.
- **s_time**
 - SSL Connection Timer
- **sess_id**
 - management of session data of SSL/TLS connections

Course master in e-business and ICT security

2011

Standard openssl commands (8)

- **smime**
 - processing of S/MIME messages
- **speed**
 - measure the performance of cryptographic algorithms
- **verify**
 - verifies/validates the X.509 digital certificates
- **version**
 - information on the version of the OpenSSL library
- **x509**
 - management of X.509 digital certificates

Course master in e-business and ICT security

2011

Preliminary steps: BASE64

- base64 encoding and decoding
 - Base 64 is an encoding/decoding system of binary data that uses 64 symbols.
 - commonly used when there is a need to encode binary data that needs be stored and transferred over media that are designed to deal with textual data. This is to ensure that the data remains intact without modification during transport
 - Is used mainly to encode/decode binary data in the e-mail messages to/from ASCII format.

Course master in e-business and ICT security

2011

BASE64 encoding algorithm

- splits the binary file in groups of 6 bits each, each of which can contain consequently values from 0 to 63.
 - each possible Base 64 value (Index) is converted in ASCII character, according to the Base 64 conversion table
 - if the total number of bits is not a multiple of 6, there are inserted null bits (0) at the end, and in the encoding it is inserted the symbol '=' for each missing group of bits.

Text content	M						a						n					
ASCII	77						97						110					
Bit pattern	0	1	0	0	1	1	0	1	0	0	1	1	0	0	1	0	1	1
Index	19						22						5					
Base64-encoded	T						W						F					

Course master in e-business and ICT security

2011

Conversion table : BASE64

Conversion table: value base 64 → ASCII character

Base 64 Value → ASCII					
0→A	11→L	22→W	32→g	43→r	54→2
1→B	12→M	23→X	33→h	44→s	55→3
2→C	13→N	24→Y	34→i	45→t	56→4
3→D	14→O	25→Z	35→j	46→u	57→5
4→E	15→P	26→a	36→k	47→v	58→6
5→F	16→Q	27→b	37→l	48→w	59→7
6→G	17→R	28→c	38→m	49→x	60→8
7→H	18→S	29→d	39→n	50→y	61→9
8→I	19→T	30→e	40→o	51→z	62→+
9→J	20→U	31→f	41→p	52→0	63→/
10→K	21→V		42→q	53→1	

Course master in e-business and ICT security

2011

BASE 64

- to encode a file (data) in base 64 format:

```
$ openssl base64 -in filename.bin -out filename.b64
```

- to decode a file from the format base 64:

```
$ openssl base64 -d -in filename.b64 -out filename.bin
```

- Exercise: encode/decode the text message "Questo è un test di codifica" in/from base 64.

Course master in e-business and ICT security

2011

SYMMETRIC CRYPTOGRAPHY

Course master in e-business and ICT security

2011

Symmetric algorithms

nome	chiave	blocco	note
DES	56 bit	64 bit	obsolete
3-DES	112 bit	64 bit	
3-DES	168 bit	64 bit	
IDEA	128 bit	64 bit	
RC2	8-1024 bit	64 bit	
RC4	variabile	stream	
RC5	0-2048 bit	1-256 bit	
AES	128-256 bit	128 bit	alias Rijndael
Blowfish	32-448 bit	64 bit	
CAST-128	40-128 bit	64 bit	

Course master in e-business and ICT security

2011

Symmetric crypto: exercises

- to encrypt/decrypt data with openssl tool you can use the openssl commands:

- enc, des3, bf, rc2, rc4, ... (\$ man enc)

- exercise: encrypt a file *ptext* with AES (128) in CBC mode, save the encrypted message in the file *ctxt*:

```
$ openssl enc -e -in ptext -out ctxt -aes-128-cbc -nosalt -p
```

- OpenSSL generates a symmetric key and an initialization vector (iv) from the password inserted by the user, for example:

```
$ enter aes-128-cbc encryption password:
$ Verifying - enter aes-128-cbc encryption password:
$ key=E358EFA489F58062F10DD7316B65649E
iv =8A222106B38147AB215285A3EEB8990B
```

Course master in e-business and ICT security

2011

Symmetric crypto: exercises (2)

- Exercise: execute again:

```
$ openssl enc -e -in ptext -out ctxt -aes-128-cbc -nosalt -p
```

- now execute (at least two times):

```
$ openssl enc -e -in ptext -out ctxt -aes-128-cbc -p
```

- what do you observe ?

- enc can generate a key (and IV) from a password
 - the password must be recorded by the user, it provides weak security (is not random), insufficient as a key (e.g. subject to dictionary attacks)
 - enc calculates the hash of the password concatenated with a "salt"

Course master in e-business and ICT security

2011

Some terms

- Known-plaintext attack:

- The cryptanalyst knows **one or several pairs of ciphertext and the corresponding plaintext**.
 - The goal is to find the key used to encrypt these messages or a way to decrypt any new messages that use that key.

- Dictionary attack:

- technique for defeating a cipher or authentication mechanism by trying to determine its decryption key or passphrase by searching likely possibilities.
 - uses a targeted technique of successively trying all the words in an exhaustive list called a dictionary (from a pre-arranged list of values).

Course master in e-business and ICT security

2010

Symmetric crypto: exercises (3)

- salt = random value and public (serves in decryption phase)
 - avoids known plaintext attacks
 - provides freshness: the same password, different keys at each encryption
- further details: function EVP_BytesToKey in crypto/evp/evp_key.c; check out http://ftp.rsasecurity.com/pub/pkcs/pkcs-5v2/pkcs5v2_1.pdf for a detailed description of "salt" usage
- Exercise: decrypt the ciphertext *ctxt* generated above:

```
$ openssl enc -d -in ctxt -out dtext -K chiave -iv iv -aes-128-cbc -p
```

- note the option `-d` (instead of `-e`), and that the name of files has changed

Course master in e-business and ICT security

2011

Symmetric crypto: exercises(4)

- What do the following commands?

```
$ openssl enc -e -des3 -salt -in plaintext.txt -out ciphertext.bin
$ openssl enc -e -des3-ede-ffb -d -in ciphertext.bin -out plaintext -pass pass:corosicurezza
$ openssl bf-cfb -salt -in plaintext -out ciphertext.bin -pass pass:diana
$ openssl rc5 -in plaintext.txt -out ciphertext.bin -S C62CB1D49F158ADC -iv E9EDACA1BD7090C6 -K 89D4B1678D604FAA3DBFFD030A3114B29
```

Course master in e-business and ICT security

2011

HASH

Course master in e-business and ICT security

2011

Hash algorithms

name	block	digest	definition	note
MD28 bit	128 bit	RFC-1319	obsolete	
MD4	512 bit	128 bit	RFC-1320	obsolete
MD5	512 bit	128 bit	RFC-1321	good
RIPEMD	512 bit	160 bit	ISO/IEC 10118-3	optimum
SHA-1	512 bit	160 bit	FIPS 180-1 RFC-3174	good
SHA-224	512 bit	224 bit	FIPS 180-2	optimum
SHA-256	512 bit	256 bit	FIPS 180-2	optimum
SHA-384	512 bit	384 bit	FIPS 180-2	optimum
SHA-512	512 bit	512 bit	FIPS 180-2	optimum

Course master in e-business and ICT security

2011

HASH: Commands + Exercises (1)

- to calculate a digest with openssl tool, you can use the openssl commands:
 - dgst, sha, sha1, md2, md4, md5, rmd160
- Exercise: calculate a digest with SHA1 algorithm on a file plaintext.txt (and print out on stdout the result in hexadecimal):

```
$ openssl sha1 plaintext.txt
$ openssl dgst -sha1 plaintext.txt
```

- calculate a digest with the algorithm SHA-256 and SHA-512 on a file plaintext.txt:

```
$ openssl dgst -sha256 plaintext.txt
$ openssl dgst -sha512 plaintext.txt
```

Course master in e-business and ICT security

2011

HASH: Exercises (2)

- Exercise: calculate a hash with SHA1 on the file plaintext.txt and save the hash in the file digest.txt:

```
$ openssl sha1 -out digest.txt file.txt
```

Course master in e-business and ICT security

2011

ASYMMETRIC CRYPTOGRAPHY

Course master in e-business and ICT security

2011

Asymmetric crypto: exercises (1)

- useful openssl commands:
 - genrsa, rsa, rsautil, gendsa, dsa, ..
- Exercise: generate a 2048 bit RSA key pair:

```
$ openssl genrsa -out rsa.key 2048
```

- public key**
 - modulus: N (arithmetic mod N)
 - publicExponent: $e = 2^{16} + 1$ (enc, verif)
 - is fixed
- private key**
 - privateExponent: d (dec, sign)
 - prime*: p, q ($N = pq$)
 - exponent*, coefficient: to speed up operations
 - standard PKCS1

Course master in e-business and ICT security

2011

RSA Encryption

- extract the public key from the file rsa.pubkey and view its content:

```
$ openssl rsa -in rsa.key -pubout -out rsa.pubkey
$ openssl rsa -pubin -in rsa.pubkey -text
```

- Exercise: create a message plain encrypt it with the RSA public key (2048-bit):

```
$ openssl rsautl -encrypt -in plain -pubin -inkey rsa.pubkey -out ctextrsa -pkcs
```

- in practice, the maximum length of the message to encrypt is : $N' = N - 11$ (where N is the length in bytes of the RSA modulus). For example, for 2048-bit, $N' = 245$. See PKCS #1 v2.1 (RFC 3447).

- decrypt the message ctextrsa:

```
$ openssl rsautl -decrypt -in ctextrsa -inkey rsa.key
```

Course master in e-business and ICT security

2011

Asymmetric crypto : Examples (2)

- sign the content of the file `plain` with the RSA private key (`rsa.key`); the signature is saved in `sig.bin`:

```
$ openssl rsautl -sign -in plain -inkey rsa.key -out sig.bin
```

- verify the signature with the RSA public key (`rsa.pubkey`) and save the original data in the file `prova_sig.txt`:

```
$ openssl rsautl -verify -in sig.bin -pubin -inkey rsa.pubkey -out prova_sig.txt
```

Course master in e-business and ICT security

2011

Length of an RSA signature

- length of the signature is proportional to the length of the key:
 - the digest has a fixed dimension
 - 16 bytes for MD5
 - 20 byte for SHA-1 and RIPEMD
 - the signature instead is long as the modulus (N) used:
 - 64 bytes for keys of 512 bits
 - 128 bytes for keys of 1024 bits
 - 256 byte for keys of 2048 bits

Course master in e-business and ICT security

2011

X.509 v3 DIGITAL CERTIFICATES

Course master in e-business and ICT security

2011

Syntax of a certificate (ASN.1)

```
Certificate ::= SEQUENCE {
  tbsCertificate TBSCertificate,
  signatureAlgorithm AlgorithmIdentifier,
  signatureValue BIT STRING
}
```

```
TBSCertificate ::= SEQUENCE {
  version [0] Version DEFAULT v1,
  serialNumber CertificateSerialNumber,
  signature AlgorithmIdentifier,
  issuer Name,
  validity Validity,
  subject Name,
  subjectPublicKeyInfo SubjectPublicKeyInfo,
  issuerUniqueID [1] IMPLICIT UniqueIdentifier OPTIONAL,
  -- if present, version must be v2 or v3
  subjectUniqueID [2] IMPLICIT UniqueIdentifier OPTIONAL,
  -- if present, version must be v2 or v3
  extensions [3] Extensions OPTIONAL
  -- if present, version must be v3
}
```

Course master in e-business and ICT security

2011

Commands + Exercises (1)

- to create a demo CA, you need to execute the script:

```
$ /usr/local/ssl/misc/CA.pl -newca
```

```
demoCA
├── certs
├── newcerts
├── crl
├── private
├── index.txt
├── serial
├── cacert.pem
└── cakey.pem
```

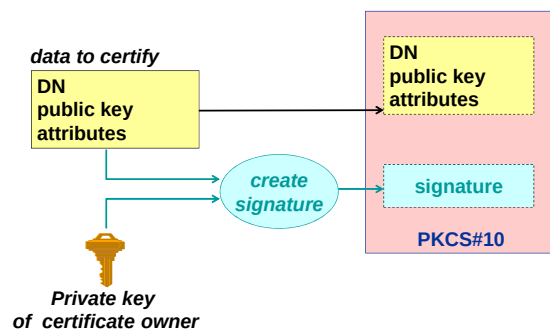
- to request a new certificate:

```
$ openssl req -newkey rsa:2048 -pubkey -keyout usersakey.pem -out usercertreq.pem
```

Course master in e-business and ICT security

2011

Certificate request format: PKCS-10



Course master in e-business and ICT security

2011

Digital certificates: Exercises (2)

- view a certificate request:

```
$ openssl req -in usercertreq.pem -text -noout
```

- issue an X.509 v3 certificate:

```
$ openssl ca -in usercertreq.pem -out user_cert.pem
```

- view the content of an X.509 certificate in PEM format:

```
$ openssl x509 -in user_cert.pem -text
```

- convert the certificate from PEM format to DER format:

```
$ openssl x509 -inform PEM -in user_cert.pem -outform DER -out user_cert.der
```

- view the content of an X.509 certificate in DER format:

```
$ openssl x509 -inform DER -in user_cert.der -text
```

Course master in e-business and ICT security

2011

Digital certificates: Exercises (3)

- verify an X.509 certificate:

```
$ openssl verify -CAfile demoCA/cacert.pem user_cert.pem
```

- revoke a certificate:

```
$ openssl ca -revoke demoCA/newcerts/<serial>.pem
```

- issue the CRL that contains the revoked certificate:

```
$ openssl ca -gencrl -out crl.pem
```

- view the content of a CRL:

```
$ openssl crl -in crl.pem -text
```

- export a certificate:

```
$ openssl pkcs12 -export -in user_cert.pem -inkey user_pkey.pem -name "MY CERTIFICATE" -certfile demoCA/cacert.pem -out user_cert.p12
```

Course master in e-business and ICT security

2011

Digital certificates: Exercises (4)

- import the certificate in the browser:

- In SeaMonkey: Edit->Preferences->Privacy&Security->Manage Certificates->Import. Import the file user_cert.p12
- In IE: Tools -> Internet Options -> Content -> Certificates -> Import. Import user_cert.p12
 - You should view the demoCA among the trusted CA

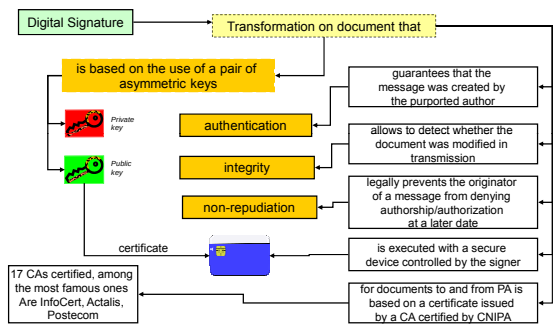
- control the integrity of the certificate:

- view the field SHA1 fingerprint of the certificate user_cert imported in the browser: serves to control the integrity of all the fields of the certificate (especially the public key !!)
 - In practice if the user wants to control the integrity of a certificate he could phone the CA and asks the value of the fingerprint of that certificate. If the fingerprint is the same, then this guarantees the integrity of the certificate (in all his parts)
 - the fingerprint is calculated as the hash of the certificate (in binary), like for example by executing:
 - \$ openssl sha1 user_cert.der

Course master in e-business and ICT security

2011

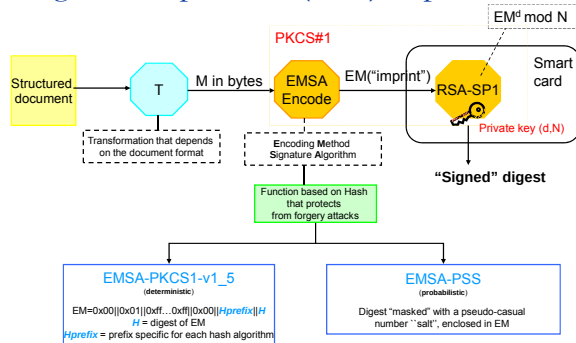
Digital signature in PA (Italy)



Course master in e-business and ICT security

2011

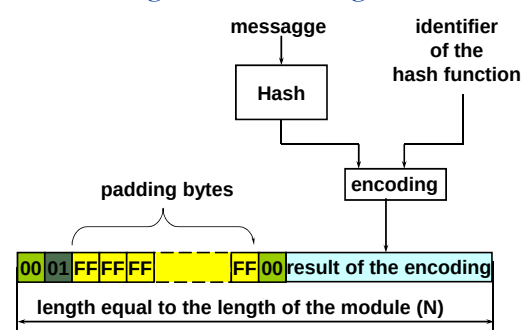
Signature operation (RSA) in practice



Course master in e-business and ICT security

2011

Flow diagram of RSA signature



Course master in e-business and ICT security

2011

